

Page Navigation

- Anatomy of a Level
 - Actor Block
 - Components Block
 - Type 0 - SceneComponent
 - Type 1 - StaticMeshComponent
 - Type 2 - SkeletalMeshComponent
 - Type 3 - PointLightComponent
 - Type 4 - SpotLightComponent
 - Type 5 - DirectionalLightComponent
 - Type 6, 7 and 8 - BoxComponent, ...
 - Type 9 - SpringArmComponent
 - Materials

Book Navigation

- Unreal Level Exporter
- Usage for Level Designers
- Usage for Programmers

Usage for Programmers

As a programmer the main way you'll be exposed to this tool is through the JSON file it generates as well as any resource files that come along with it depending on export settings. It's imperative that you understand the format and this document is meant to help you with this and also aims to provide some general guidelines around working with the data.

Anatomy of a Level

When exporting a level the resulting JSON file will contain a JSON array of Actor objects (Game Objects). Each Actor contains relevant information such as name, type, transform, etc as well as a list of Components. Exactly what is contained in each component varies depending on its type. An example JSON file for a level looks something like this:

```
► Example Level JSON
```

This may initially seem overwhelming but if you spend a moment to read the data, and continue reading this document, it should prove to be pretty straight forward.

Actor Block

The primary object in this array is an Actor. An Actor represents something in the game Scene or World. The following table shows the primary fields that you may encounter and some suggestion on how to use them in your own Game Engine.

Field	Description
Name	The unique name of this Actor. This is unique to the Level the Actor is part of. You can use this for unique identification in an <code>unordered_map</code> of <code>string-to-Actor*</code> , for example.
Archetype	This is the class name of this Actor which can be either a C++ class or a Blueprint Class. Blueprint-based classes always end in <code>_C</code> while C++ classes do not. It represents the template used for this Actor and consequently can be used to identify what object you should instantiate in your own Engine. Exactly how you identify this depends on your structure. You may perhaps use an overarching Actor or GameObject class and just add Components, or perhaps you have another <code>unordered_map</code> of Archetype-To-Class, or perhaps an enum to construct the correct object.
Transform	A Transform Matrix in float-array format. There are 16 floats ordered in rows (the first 4 floats is Row 1, next 4 are Row 2 and so on). This Matrix is in Unreal Engine format and must be converted to work with your own DirectX-based engines. See the Space Conversion section for details.
Tags	These are string values which are set by the Level Designer. They can contain anything and are always in the same case as typed in the editor. This is a good way to pass arbitrary information on objects such as setting flags or passing properties. Instruct your Level Designer to write something like <code>"IsDestructible=true"</code> as a tag and parse it accordingly in your engine.
Components	A JSON array of Component Objects. See the Components section below for more information.

Components Block

Here you will find any number of Components an Actor has. All actors will have at least one Component because Unreal Engine requires this. Actors have no Transform of their own in UE --- this is represented by their Root Component which also controls their appearance. The Exporter will however differentiate this to make parsing easier.

A Component can be quite a few different things; Meshes, Lights, Particle Systems and so on. To limit the work require the exporter only writes certain types of Components to the JSON file. These are differentiated by a Type property to make identification and processing easier.

All components have the following fields:

Field	Description
Name	Actor-Unique name for this Component. Can be used to identify a specific Component.
TypeID	An integer value to indicate which Component type this is. See the list below these fields for more information. Note: A type of -1 means that this is a custom component using custom serialization. Highly recommend making a <code>vector</code> , <code>unordered_map</code> or <code>enum</code> to determine what to do in your own Engine.
Parent	The name of the Parent Component if there is one. If this is the Root Component Parent will have an empty value.
Transform	A Transform Matrix in float-array format. There are 16 floats ordered in rows (the first 4 floats is Row 1, next 4 are Row 2 and so on). This Matrix is in Unreal Engine format and must be converted to work with your own DirectX-based engines. See the Space Conversion section for details. This Transform is relative to the owning Actor. The Root Component will always have an Identity Transform.
Tags	These are string values which are set by the Level Designer. They can contain anything and are always in the same case as typed in the editor. This is a good way to pass arbitrary information on objects such as setting flags or passing properties. Instruct your Level Designer to write something like <code>"IsDestructible=true"</code> as a tag and parse it accordingly in your engine.

Components will also have specific properties depending on their type and these may in turn contain other data blocks such as Material data.

Type 0 - SceneComponent

A Scene Component is just a Transform. It helps provide relative structure inside an Actor in Unreal Engine such as moving the transform center of its child components. This component type has no extra fields. It is also the default Root Component in an Actor in Unreal Engine unless it has been replaced by the Level Designer.

Type 1 - StaticMeshComponent

A Component which contains a single, static Mesh. This is typical for Level Geometry, Props, Doors, etc. The StaticMeshComponent has these fields:

Field	Description
Mesh	The asset name of the Mesh used by this Component.
ContentPath	Where this Asset is located relative to the Unreal Project. This is always in the format <code>/Path/To/Asset.Asset</code> .
Materials	An array of Material Blocks used by the Mesh Asset. These Materials are always written in the order of usage, i.e. the first Material is used for Material Slot 0 on the Mesh and so on.

Type 2 - SkeletalMeshComponent

A Component which contains a Skeletal Mesh, i.e. a Mesh with skinning and joints. This component is handled in exactly the same way as the StaticMeshcomponent.

Type 3 - PointLightComponent

Represents a typical Point Light. This component has the following fields:

Field	Description
Color	A float-array with RGBA describing the lights color.
Intensity	Light Intensity in Unreal Units. Requires the same conversion ratio as discussed in the TGP Course.
FalloffExponent	The Exponent used for Light Falloff. This is for advanced light setup uses and not something which is handled in TGP course (but which is easy enough to change). By default this exponent should represent Inverse-Square Law.
AttenuationRadius	The lights Attenuation or Falloff radius in centimeters.

Type 4 - SpotLightComponent

Represents a typical Spot Light. This shares the same fields as Point Light Component but has these additional fields:

Field	Description
InnerConeAngle	The angle of the spotlight inner falloff cone in degrees.
OuterConeAngle	The angle of the spotlight outer falloff cone in degrees..

Type 5 - DirectionalLightComponent

A Directional Light. Should only appear on Actors in extreme cases except for the DirectionalLightActor. This Component will only have the Color and Intensity fields.

Type 6, 7 and 8 - BoxComponent, SphereComponent and CapsuleComponent

Represents a 3D Collider. Can be used to provide the bounds of an object for simple collision usage. Not intended to replace Graphical Artist-generated collisions but can instead be used to define triggers in different shapes such as for Buttons, Doors, Traps, etc.

These components define the following fields:

Field	Description
Bounds	The Bounds Maxima in XYZ for a Box Component. Has no meaning for other shapes.
Radius	The Radius of a Sphere or Capsule Component. Has no meaning for Box Components.
HalfHeight	The Half Height of a Capsule (i.e. height from Origin). Has no meaning for other shapes.

Type 9 - SpringArmComponent

In Unreal Engine you can use a Spring Arm Component to provide advanced camera positioning for an Actor such as an offset Orbit Cam or similar. It acts as a real world Camera Boom arm--a long stick to which a Camera is mounted. This is provided for specific circumstances and compatibility.

The Spring Arm Component has these fields:

Field	Description
SocketOffset	An offset applied to the end of the spring arm to which the Camera (or another object) is attached.
ArmLength	The lenght of the camera boom arm.

Materials

Materials are also sparsely stored in the JSON file, showing which textures and settings are used. For Material Parameters the Type denotes the data type of the parameter. This is a direct write of UE's EMaterialParameterType enum so it contains quite a few different values which will probably never appear in your game projects. Recommendation is to support these on an as-needed basis with those marked with an asterisk* being the minimum implementation.

Type	Data Type
0	Scalar (float)*
1	Vector3 Float*
2	Vector3 Double
3	Texture*
4	TextureCollection
5	Font
6	RuntimeVirtualTexture
7	SparseVolumeTexture
8	StaticSwitch (bool)
9	ParameterCollection

Materials also export their Domain, Shading Model and Blend Mode:

Domain	Shading Model	Blend Mode
0 Surface	0 Unlit	0 Opaque
1 DeferredDecal	1 DefaultLit	1 Masked
2 LightFunction	2 Subsurface	2 Translucent
3 Volume	3 PreintegratedSkin	3 Additive
4 PostProcess	4 ClearCoat	4 Modulative
5 UI	5 SubsurfaceProfile	5 AlphaComposite
6 RVT	6 TwoSidedFoliage	6 AlphaHoldout
	7 Hair	7 Col. Translucent Transmittance
	8 Cloth	8 Grey Translucent Transmittance
	9 Eye	9 Col. Transmittance
	10 SingleLayerWater	
	11 ThinTranslucent	
	12 Strata	

Details

- Revision #16
- Created 1 year ago by Daniel Borgshammar
- Updated 2 weeks ago by Daniel Borgshammar
- Referenced by 1 item

Actions

- Revisions
- Export